

---

# Práctica 4: Iluminación local

---

**Objetivo:** Cálculo de luces

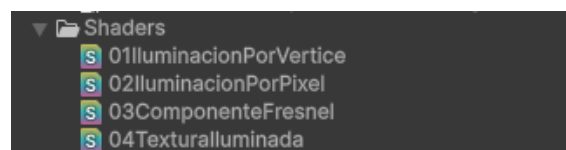
## 1. Instrucciones

[Iguales a las de la práctica anterior]

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo shaders y materiales que se aplican sobre un cubo.

La entrega debe incluir únicamente los shaders individuales con los nombres *de ficheros* concretos indicados en el texto y una captura de la pestaña *Game* del editor de Unity en la que se vea el cubo con el material aplicado. Es importante que los nombres de los ficheros entregados coincidan con los pedidos. En el enunciado también se especifica qué nombre deben tener las variables HLSL de los parámetros del material.

Si vas creando los shaders en la carpeta Shaders del proyecto, al final de la práctica deberías ver algo así:



En la entrega habrá un fichero con extensión `.shader` por cada uno de los archivos que aparecen en la imagen, así como una captura del resultado de aplicar ese material con extensión `.png` o `.jpg`. Por ejemplo, se espera un fichero `01IluminacionPorVertice.shader` y un `01IluminacionPorVertice.png` (o `.jpg`).

Para el desarrollo se recomienda crear la escena base con el cubo y duplicarla de forma que en cada escena el cubo utilice un material y shader distinto.

## 2. Punto de partida

[**NO ES IGUAL** al de la práctica anterior]

Crema un proyecto 3D en Unity que utilice el *Universal Render Pipeline* con una escena con:

- El contenido por defecto de la plantilla de Unity, con skybox, cámara y luz direccional.
- La cámara con la localización (L), escalado (S) y rotación (R) siguientes: L:  $(-3, 1, -3)$ , S:  $(1, 1, 1)$ , R:  $(10, 45, 0)$ .
- **[Cambiado]** Un cubo en L:  $(1, 0, -1)$ , S:  $(1, 1, 1)$ , R:  $(0, 0, 0)$ .
- **[Nuevo]** Una esfera en L:  $(-1, 0, 1)$ , S:  $(1, 1, 1)$ , R:  $(0, 0, 0)$ .

Para cada shader y material, aplicalo a ambas geometrías antes de hacer la captura.

### 3. Iluminación local por vértice

Crea un shader `01IluminacionPorVertice` que implemente la iluminación *por vértice* incorporando luz de ambiente, difusa y especular.

Recuerda que el *shader* debe tener tanto el parámetro con el color base como el relacionado con la luz especular:

```
Properties
{
    _Color ("Color", Color) = (1.0, 0.0, 0.0)
    _GlossPower ("Gloss Power", Float) = 100
}
```

### 4. Iluminación local por píxel

Basándote en el anterior, crea un nuevo shader y material, `02IluminacionPorPixel` que implemente la misma iluminación pero calculada en cada píxel.

Tendrás que mover el código relacionado con la iluminación al PS dejando en el VS el cálculo de la normal del vértice (en WS) y el vector entre la cámara y el vértice (también en WS). **No olvides normalizar de nuevo ambos valores** en el PS.

### 5. Fresnel

Algunos modelos de iluminación incorporar un componente adicional conocido como *luz de Fresnel* que coge importancia cuando el objeto se ve de perfil. En concreto, el componente fresnel es máximo cuando el ángulo entre la normal y la vista son perpendiculares.

La fórmula es:

$$L_{fresnel} = L_{color} \times ((1 - n) \cdot v)^\beta$$

donde  $L_{color}$  es el color de la luz y  $n$  y  $v$  son la normal de la superficie y el vector vista respectivamente.  $\beta$  es el “coeficiente de fresnel” del material que deberás incorporar como parámetro al shader:

```
Properties {
    ...
    _FresnelPower ("Fresnel Power", Float) = 10
}
```

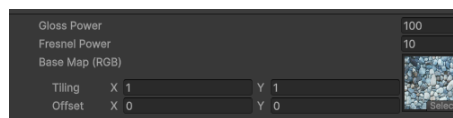
## 6. Textura iluminada

Crea un nuevo shader y material, `04TexturaIluminada`, que sustituya en el anterior el parámetro con el color por una textura.

Properties

```
{
    _GlossPower("Gloss Power", Float) = 100
    _FresnelPower("Fresnel Power", Float) = 10
    _BaseMap("Base Map (RGB)", 2D ) = "" {}
}
```

Busca en la documentación de Unity cómo añadir soporte para los campos Tiling y Offset e incorporalo en el shader.



## 7. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Los ficheros de shaders y capturas indicados anteriormente con los nombres exactos.
- Incluir además un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.